sas-pme — Roadmap des 34 prochaines heures

Hackathon Track 3 « Open Track » · IA × cybersécurité défensive · équipe A · B · C · H0 = début du chantier

> **Le pitch.** Un sas de sécurité posé dans la PME retient chaque nouvelle version de paquet Python le temps qu'une IA locale l'examine et lise ce qu'Internet en dit, puis surveille les connexions sortantes. Si un paquet piégé passe quand même, le sas le démasque dès qu'il appelle son serveur.
> **« Le sas qui fait attendre les mises à jour, pas vos développeurs. »**

Le défenseur aidé

Le responsable technique d'une PME ou startup qui développe en Python (5 à 100 personnes), sans RSSI ni SOC. Vente par l'infogérant.

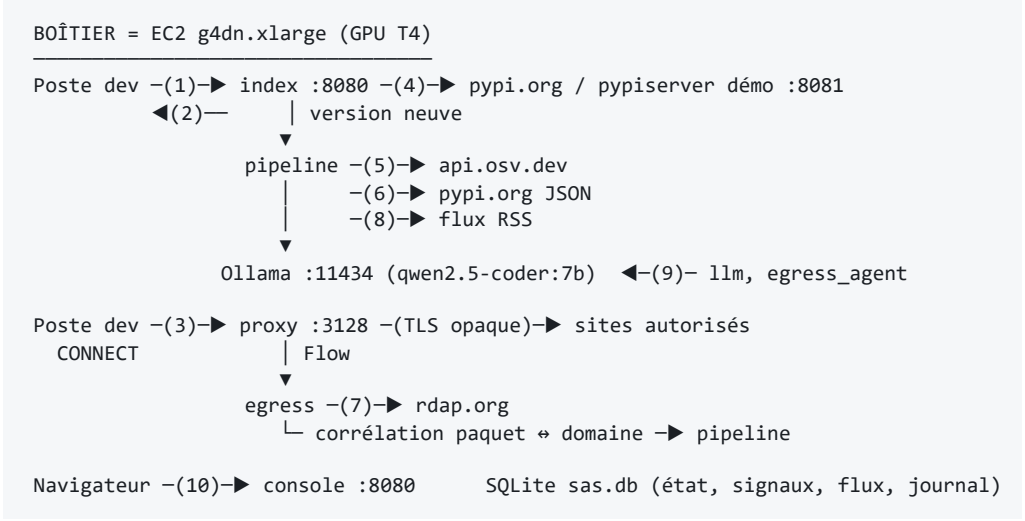
Principes non négociables

Le code décide, l'IA alerte	un LLM peut monter un risque, jamais l'abaisser ni approuver
Un texte lu n'est pas un ordre	code, changelog, article : des données à juger (anti-injection)
Tout local	LLM sur le boîtier (Ollama) ; aucun service cloud managé
On lit Internet, on ne lui parle pas de vous	ne sortent que nom de paquet + version, et des domaines
Transparent	une ligne de config ; pip ne voit jamais d'erreur due à la quarantaine

Ce que fait le sas

Barrière	Comment
1. Index PyPI filtré	pip ne voit que les versions approuvées ; une version neuve part en quarantaine (48 h), pip prend la précédente sans erreur
2. Analyse du paquet	étage 1 : typosquat, âge, mainteneurs, OSV, OpenSSF · étage 2 : diff avec la version précédente (setup.py, exec, base64, secrets, domaines) · étage 3 : LLM local, verdict JSON + phrase en français
3. Veille	toutes les 2 h : OSV, statut PyPI (yanked), blogs de chercheurs ; un article peut bloquer, jamais débloquer
4. Sorties	proxy CONNECT sans déchiffrement : domaine nouveau, domaine jeune, IP brute, battement ; **corrélation paquet installé ↔ domaine contacté** → blocage des deux
5. Console + journal	une phrase et ses preuves par décision ; approuver/bloquer en un clic ; journal chaîné par hash (NIS2, CRA)

Architecture : machines, logiciels, données en transit



Logiciels sur le boîtier

Logiciel	Rôle	Port
Ubuntu 22.04, AMI Deep Learning (pilote NVIDIA)	système	—
Python 3.12 · FastAPI · uvicorn	index filtré + console + tâches de fond	8080
Proxy CONNECT maison (asyncio)	relais des sorties, comptage, blocage	3128
Ollama + qwen2.5-coder:7b	LLM local (verdicts, articles, résumés)	11434 local
SQLite	état, signaux, flux, journal chaîné	fichier
pypiserver	paquets piégés + flux RSS de démo	8081 local
httpx · pydantic · rapidfuzz · packaging · feedparser · Jinja2 · HTMX · pytest · respx	HTTP, schémas, typosquat, versions, RSS, pages, tests	—

Données en transit

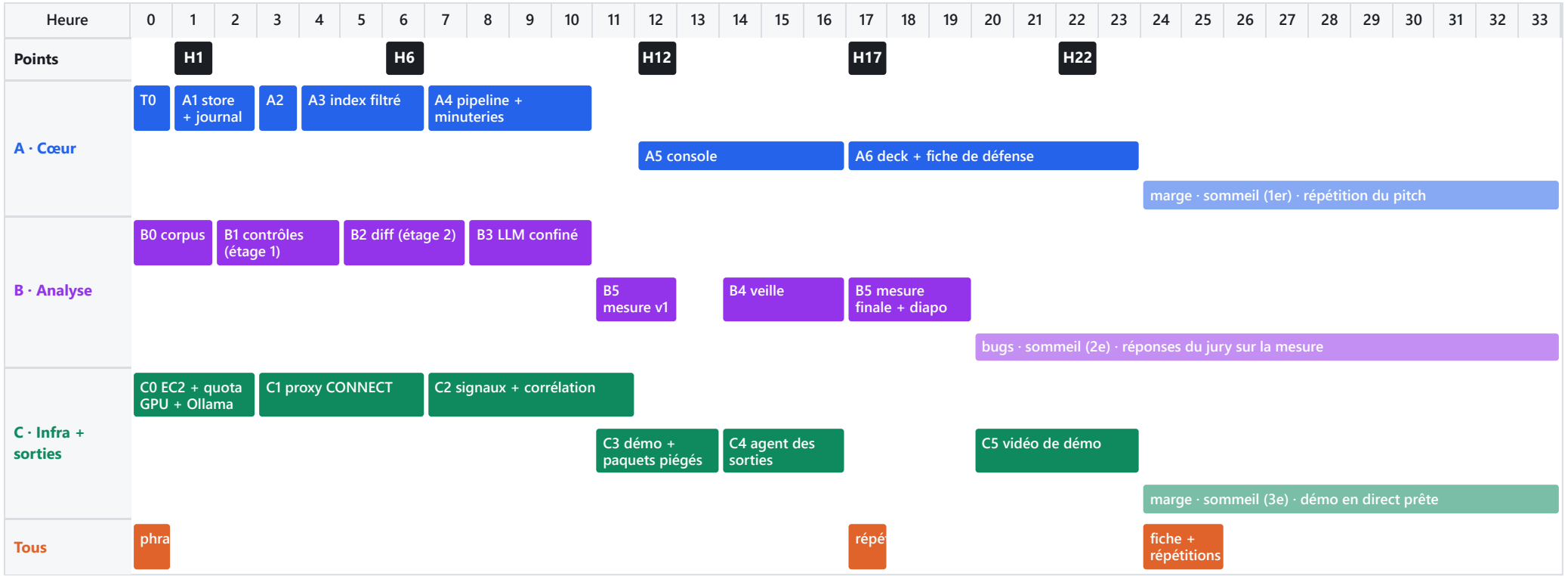
#	De → vers	Protocole	Contenu
1	poste → index	HTTP GET `/simple/<p>/`, `/files/…`	nom du paquet ; IP source = poste
2	index → poste	HTML PEP 503 + archive	versions visibles seulement, `#sha256` vérifié
3	poste → proxy	`CONNECT host:443` + TLS opaque	sas lit domaine, port, heure, octets, durée — **jamais le contenu**
4	index → amont	HTTPS, JSON PEP 691	nom ; fichiers publics
5	pipeline → OSV	HTTPS POST `/v1/query`	nom + version ; avis CVE/MAL
6	pipeline → PyPI JSON	HTTPS GET `/pypi/<n>/<v>/json`	nom + version ; date, mainteneurs, yanked
7	egress → RDAP	HTTPS GET	un domaine ; date de création
8	veille → RSS	HTTPS GET	rien envoyé ; articles XML
9	sas → Ollama	HTTP local POST `/api/chat`, `format=`schéma, temp. 0	constats, extraits ≤ 200 car., changelog, articles ; JSON validé pydantic
10	navigateur → console	HTTP + HTMX	états, signaux, décisions humaines

> **Ne quitte jamais le boîtier :** code de la PME, liste complète de ses dépendances, identité des postes, contenu des connexions, verdicts. **Sort :** nom + version (5, 6), domaines (7), lecture de flux publics (8).

Configuration d'un poste (une fois)

```
pip config set global.index-url http://<ip-du-sas>:8080/simple
setx HTTPS_PROXY http://<ip-du-sas>:3128
```

Planning heure par heure



Une colonne = une heure depuis le début du chantier. Barres pâles = marge et sommeil en quinconce (une personne dort pendant que deux veillent). Noir = point de synchronisation de 15 min. T0 = contrats partagés ; A2 = politique ; « phrase » = phrase-problème envoyée aux organisateurs.

Points de synchronisation et règles de coupe

Heure	Ce qui doit marcher	Si non
H1	`models.py` commité ; EC2 lancée ou quota GPU demandé ; phrase-problème envoyée	A pousse les contrats avant tout le reste
H6	`pip install` depuis un PC passe par l'index du sas sur l'EC2	C aide A sur l'index ; le proxy glisse d'une heure
H12	mesure configuration 1 de bout en bout ; proxy relaie ; LLM répond en < 5 s	**on coupe la veille (B4)** ; modèle `3b` si lent
H17	démo complète répétée une fois	gel des fonctions, plus que des corrections
H22	deck, mesure finale, vidéo enregistrée	la vidéo passe avant le deck

Tâches par personne

A Cœur · console · pitch

☐ **T0 (H0–H1)** `pyproject.toml`, `models.py` : State, Level, Source, Signal, Finding, Flow, HumanDecision

☐ **A1 (H1–H3)** `Store` SQLite + `Journal` chaîné SHA-256, `verify()`

☐ **A2 (H3–H4)** `policy.decide()` fonction pure ; invariant « llm/article n'assouplit jamais » sur 200 cas

☐ **A3 (H4–H7)** `Upstream` PEP 691 ; `/simple` filtré, `/files` avec hash ; fail-closed ; adoption initiale ; test `pip download` réel

☐ **A4 (H7–H11)** `Pipeline` : file asyncio, étages 1→2→3, `tick` 60 s, veille 2 h, LLM absent → à revoir

☐ **A5 (H12–H17)** console HTMX : compteurs, fil, fiche version, fiche poste, journal, approuver/bloquer

☐ **A6 (H17–H24)** deck 8 diapos + fiche de défense (10 questions du jury)

B Analyse · mesure

☐ **B0 (H0–H2)** corpus : 30 malveillants (OpenSSF malicious-packages), 50 mises à jour saines. **Jamais installer ni exécuter un échantillon.**

☐ **B1 (H2–H5)** `checks.run_fast / run` : typosquat (rapidfuzz, top 5000), âge, mainteneurs, sdist seul, OpenSSF, OSV

☐ **B2 (H5–H8)** `diffscan.scan` : lignes ajoutées seulement ; setup.py/.pth/__init__, exec/eval/base64, secrets, domaines, entropie ; archives bornées

☐ **B3 (H8–H11)** `llm.judge_release / classify_article / summarise_host` : Ollama, schéma JSON, balises `<donnee>`, test d'injection

☐ **B5 v1 (H11–H13)** mesure configuration 1 — **point de contrôle H12**

☐ **B4 (H14–H17)** veille : OSV, yanked, RSS ; un article ne débloque jamais ; seules sortent nom + version

☐ **B5 final (H17–H20)** trois colonnes détection / fausses alertes + diapo

C Infra · sorties · démo

☐ **C0 (H0–H3) quota GPU « G and VT » dès H0** ; g4dn.xlarge eu-west-3, AMI Deep Learning ; Ollama + `qwen2.5-coder:7b`, verdict < 5 s ; service systemd. Repli : c7i.4xlarge + modèle 3b

☐ **C1 (H3–H7)** `serve_proxy` : CONNECT seul, relais et comptage, domaines bloqués, liste fermée CI

☐ **C2 (H7–H12)** `Egress.on_flow` : domaine nouveau, RDAP < 30 j, IP brute, battement, **corrélation 10 min**

☐ **C3 (H11–H14)** démo : `colorama-utils` 1.0/1.1 piégée, `reqeusts`, flux RSS (vrai + faux article), serveur de battement, `script.md`

☐ **C4 (H14–H17)** agent des sorties : résumé français par poste, aucun signal effacé

☐ **C5 (H20–H24)** vidéo de démo ≤ 3 min, plan de secours si le direct tombe

La démo (3 minutes)

1	`pip install` courant via le sas : rien de visible — *« transparent »*
2	une version piégée sort : pip ne la voit pas, la console explique en français — *« le code a lu ce que personne ne lit »*
3	un faux article « cette version est sûre » ne débloque rien — *« l'IA ne peut qu'alerter »*
4	un dev force l'installation ; en < 1 min, domaine et paquet sont reliés et bloqués — *« la deuxième barrière tient »*
5	diapo de mesure : trois colonnes chiffrées

Risques à surveiller

Quota GPU à 0 sur le compte AWS	demande dès H0 ; repli CPU + modèle 3b
« Où est l'IA ? » si la colonne 2 ne bat pas la colonne 1	mesure dès H12 ; mettre en avant incohérence changelog/code et lecture des articles
Plomberie qui mange le temps	un seul écosystème (PyPI), pas de déchiffrement TLS
Injection de prompt en démo	LLM ne peut que monter un risque ; test B3 dédié
Classement en Track 2 par le jury	insister : PME sans SOC + sorties corrélées, pas un scanner de dépendances